OVS User Manual

OVS Project Team

OVS Comprehensive User Manual (Print Edition)

1) Getting Started
- 1.1 What OVS Is
- 1.2 Deployment Modes
- 1.3 Access URLs (Typical Local Setup)
- 1.4 Environment Requirements
- 1.5 First Login and Initial State
- 1.6 High-Level Workflow Map
- 1.7 Data Handling Model
- 1.8 Recommended Reading Next

2) Roles, Permissions, and Navigation
- 2.1 Supported Account Types
- 2.2 Route Access Logic (Frontend)
- 2.3 Major Navigation Areas
- 2.4 Admin Responsibilities
- 2.5 HR Manager Responsibilities
- 2.6 Candidate Responsibilities
- 2.7 Permission Design Notes
- 2.8 Navigation Troubleshooting

3) Subscription, Billing, and Payment Methods
- 3.1 Overview
- 3.2 Typical Subscription Journey
- 3.3 Plans and Quotas
- 3.4 Billing Management in Settings
- 3.5 Important Behavior: Unsubscribe
- 3.6 Callback and Confirmation Behavior
- 3.7 Billing API Surfaces (User-Relevant)
- 3.8 Multi-Currency Notes
- 3.9 Troubleshooting Billing Flow

4) Authentication, Login, and Security
- 4.1 Authentication Endpoints
- 4.2 Login Flow (Standard User)
- 4.3 Admin Login Flow
- 4.4 Password Management
- 4.5 Profile Management
- 4.6 Two-Factor Authentication (2FA)
- 4.7 Security Page
- 4.8 Session and CSRF Notes
- 4.9 Email Delivery Behavior

5) HR Campaign and Rubric Workflows
- 5.1 Campaign Lifecycle Overview
- 5.2 Campaign Management Surfaces
- 5.3 Creating a Campaign
- 5.4 Rubric Strategy
- 5.5 Rubric Design Best Practices
- 5.6 Evaluation Workflow
- 5.7 HR Daily Workflow Example
- 5.8 Common Campaign Mistakes

6) Candidate Invitations and Access Journey
- 6.1 Invitation Model
- 6.2 Candidate Access APIs
- 6.3 Candidate Entry Points
- 6.4 Candidate Enrollment Surfaces
- 6.5 Invitation Operational Tips
- 6.6 Candidate Session Security Notes

6.7 Candidate Experience Checklist
7) Applications and Document Vetting
7.1 Application Pages
7.2 Application APIs
7.3 Document Vetting Components
7.4 Typical Case Workflow
7.5 Verification Status Interpretation
7.6 Recheck Operations
7.7 Upload Guidance
7.8 Decision Quality Best Practices
8) AI Interviews and Live Video Calls
8.1 Interview Surfaces
8.2 Interview Flow
8.3 Video Meeting APIs (LiveKit-Oriented)
8.4 Scheduling Best Practices
8.5 1v1 and 1vMany Notes
8.6 Troubleshooting Interview/Meeting Issues
9) Admin Operations and Control Center
9.1 Admin UI Areas
9.2 Admin APIs
9.3 Key Admin Responsibilities
9.4 User Management Procedures
9.5 Case Oversight Procedures
9.6 Governance Recommendations
9.7 Delegation Model
10) Monitoring, Audit Logs, and AI Runtime Health
10.1 Monitoring Surfaces
10.2 Audit Endpoints
10.3 ML Monitoring Endpoints
10.4 AI Monitor Endpoints
10.5 Runtime Health Signals
10.6 Audit Usage Pattern
10.7 Monitoring Best Practices
10.8 Troubleshooting "Unavailable Runtime" Indicators
11) Background Checks and Provider Integration
11.1 Background Check Scope
11.2 Provider Modes
11.3 Consent and Governance
11.4 Refresh and Event Flow
11.5 Webhook Security Practices
11.6 Operator Checklist
11.7 Common Background Check Issues
12) Notifications and Decision Lifecycle
12.1 Notification Surfaces
12.2 Notification Use Cases
12.3 Decision Lifecycle Model
12.4 Decision Quality Controls
12.5 Notification Hygiene
13) Operational Procedures (Docker, Commands, and Checks)
13.1 Standard Startup (Docker)
13.2 Migrations and Admin Bootstrap
13.3 Health and Integrity Checks
13.4 AI/ML Command Utilities
13.5 Logging During Operations
13.6 Release Gate Commands (Recommended)
13.7 Email Mode Validation
13.8 Production Compose Notes
14) Troubleshooting Guide
14.1 Login and Auth Problems
Symptom: "CSRF token missing" or origin check failed
Symptom: Login succeeds but redirects incorrectly
14.2 Billing/Checkout Problems
Symptom: `/billing/success` hangs on "Confirming Payment"
Symptom: "Transaction reference not found"
14.3 Reminder Runtime Unavailable

Symptom: reminder runtime card shows unavailable or fetch error
14.4 Candidate/Invitation Issues
Symptom: candidate cannot access invite link
14.5 Video Call Join Issues
Symptom: cannot join meeting
14.6 Background Check Stuck in Pending
14.7 AI Monitoring Endpoints Not Accessible
14.8 Docker Build/Startup Issues
Symptom: image build fails or service unhealthy
14.9 Last-Resort Incident Procedure
15) API Endpoint Quick Map
15.1 System and Core
15.2 Authentication
15.3 Campaigns, Candidates, Enrollments, Invitations
15.4 Applications and Documents
15.5 Interviews and Video Calls
15.6 Rubrics and Scoring
15.7 Notifications, Audit, Fraud, Monitoring
15.8 Billing and Background Checks
16) Glossary and Best Practices
16.1 Glossary
16.2 User Best Practices
For HR Managers
For Admins
For Candidates
16.3 Operational Best Practices
16.4 Security Best Practices
16.5 Decision Governance Best Practices
16.6 Manual Maintenance Policy
17) Frequently Asked Questions (FAQ)
17.1 Access and Accounts
Q: Can anyone sign up directly?
Q: Why am I redirected to 2FA after login?
Q: Why can I not see admin pages?
17.2 Billing
Q: Why does cancellation not immediately remove access?
Q: Why does callback say payment pending even after checkout?
Q: Can payment method be changed?
17.3 Campaigns and Rubrics
Q: Who should create rubrics?
Q: Can a rubric be changed after activation?
Q: Why are some cases marked for manual review?
17.4 Candidate Journey
Q: Candidate says invitation link is invalid
Q: Can candidate have a permanent account?
17.5 Interviews and Video Calls
Q: Why is meeting join blocked?
Q: What is reminder runtime health?
17.6 Monitoring and Audit
Q: Who should access AI/ML monitoring?
Q: Why audit logs matter?
17.7 Background Checks
Q: Is background check provider integration real or mock?
Q: Webhook isn’t updating status. What next?
18) Role Checklists and SOPs
18.1 Admin Daily Checklist
18.2 HR Manager Daily Checklist
18.3 Candidate Support Checklist
18.4 Billing SOP
18.5 Incident SOP: Provider Outage
18.6 Incident SOP: Authentication Failures
18.7 Incident SOP: Data Integrity Concern
18.8 Release-Day Checklist
18.9 Defense/Demo-Day Checklist

OVS Comprehensive User Manual (Print Edition)

Generated from chapter files in `docs/user-manual/`.

This edition is optimized for continuous reading and PDF export.

---

1) Getting Started

1.1 What OVS Is

OVS is a web-based vetting platform that combines:

- Process orchestration (campaigns, rubrics, case tracking),
- AI-assisted evidence analysis (documents and interviews),
- Human final decisions (approve, reject, escalate),
- Administrative control and operational monitoring.

1.2 Deployment Modes

OVS can run in:

- Local development mode (Docker Compose, localhost),
- Production mode (dedicated production compose stack and hosted services).

1.3 Access URLs (Typical Local Setup)

- Frontend: `http://localhost:3000`
- Backend API root: `http://localhost:8000/api/`
- Django admin: `http://localhost:8000/admin/`
- OpenAPI schema:
  - `http://localhost:8000/api/schema/`
  - `http://localhost:8000/api/schema/swagger-ui/`
  - `http://localhost:8000/api/schema/redoc/`
- Flower (Celery monitoring): `http://localhost:5555`

1.4 Environment Requirements

Minimum runtime dependencies:

- Python 3.11+
- Node.js 18+
- PostgreSQL
- Redis
- Docker + Docker Compose (recommended for full stack startup)

1.5 First Login and Initial State

After environment setup:

1. Open the frontend landing page.
2. Select subscription flow if onboarding a new organization.
3. Complete registration (when access is granted via subscription confirmation).
4. Log in and complete 2FA for non-candidate account types.

1.6 High-Level Workflow Map

1. Admin/HR creates campaign.
2. HR defines or attaches rubric.
3. HR adds/imports candidates.
4. Candidate receives invitation link or access route.
5. Candidate submits required evidence and interview responses.
6. AI analyses are processed asynchronously.
7. HR/Admin reviews case outputs and makes a final decision.
8. Notifications and status updates are distributed.

1.7 Data Handling Model

OVS stores and displays:

- Identity and profile information (role-scoped),
- Campaign metadata and rubric versions,
- Case/evidence analysis outputs,
- Billing and subscription records,
- Audit and operational metrics.

All major actions are designed to be traceable through logs and model outputs.

1.8 Recommended Reading Next

- Continue to Roles, Permissions, and Navigation.
- Then read either:
  - HR Campaign and Rubric Workflows, or
  - Admin Operations and Control Center.

---

2) Roles, Permissions, and Navigation

2.1 Supported Account Types

OVS user types:

- `admin` (system administrator),
- `hr_manager` (firm-side vetting operator),
- `applicant` (candidate participant).

2.2 Route Access Logic (Frontend)

The frontend enforces role-aware route guards:

- Admin-only routes: e.g. `/admin/*`, `/audit-logs`, `/ml-monitoring`, `/ai-monitor`.
- HR/Staff routes: campaigns, rubrics, applications, video calls, background checks.
- Applicant-restricted routes:
  - cannot access admin dashboards,
  - cannot access staff-only security pages.

2.3 Major Navigation Areas

Public pages:

- `/` landing page,
- `/subscribe`,

- `/login`,
- `/register`,
- `/forgot-password`,
- `/reset-password/:token`,
- billing callback routes.

Authenticated pages:

- `/dashboard`,
- `/settings`,
- `/security`,
- `/campaigns`,
- `/rubrics`,
- `/applications`,
- `/video-calls`,
- `/notifications`,
- monitoring and admin pages based on role.

2.4 Admin Responsibilities

Admins can:

- Access platform-wide dashboards and analytics.
- Manage users and high-level case oversight.
- View operational audit and monitoring surfaces.
- Control governance-sensitive areas.

Typical admin routes:

- `/admin/dashboard`
- `/admin/analytics`
- `/admin/users`
- `/admin/control-center`
- `/admin/cases`
- `/audit-logs`
- `/ml-monitoring`
- `/ai-monitor`

2.5 HR Manager Responsibilities

HR Managers can:

- Build and run vetting campaigns.
- Create and apply rubrics.
- Import and manage candidate lists.
- Monitor candidate progress and case outcomes.
- Schedule and manage video meeting workflows.

Typical HR routes:

- `/dashboard`
- `/campaigns`
- `/rubrics`
- `/applications`
- `/video-calls`
- `/background-checks`
- `/fraud-insights`
- `/notifications`

2.6 Candidate Responsibilities

Candidates typically interact through:

- invitation acceptance links,
- candidate access routes,
- application/document submission pages,
- interview session routes,
- result visibility routes.

Typical candidate entry points:

- `/invite/:token`
- `/candidate/access`
- limited application-interaction paths

2.7 Permission Design Notes

- Role checks run both in frontend route guards and backend permissions.
- UI visibility is not treated as sufficient security.
- API permissions remain authoritative for data access control.

2.8 Navigation Troubleshooting

If a user cannot see expected menu items:

1. Confirm account role in profile.
2. Confirm successful authentication and profile fetch.
3. Confirm no pending 2FA challenge state.
4. Check backend permissions or token freshness.

---

3) Subscription, Billing, and Payment Methods

3.1 Overview

OVS supports subscription onboarding and ongoing billing management from the application UI.

Supported providers:

- Stripe,
- Paystack,
- Sandbox/local management flow (in selected environments).

3.2 Typical Subscription Journey

1. User clicks **Get Started** from landing flow.
2. User selects plan and billing cycle.
3. User selects payment method/provider route.
4. Hosted checkout is opened (Stripe/Paystack).
5. User returns to callback URL:
   - `/billing/success?...`
   - `/billing/cancel?...`
6. Backend confirmation endpoint finalizes access ticket/subscription state.
7. User proceeds to login/registration flow.

3.3 Plans and Quotas

Plan limits are enforced by backend quota checks.

Examples of quota-constrained operations:

- candidate import volume,
- new candidate enrollment over plan threshold.

Quota state can be viewed via billing endpoints and related UI components.

3.4 Billing Management in Settings

From `/settings` for non-applicant users:

- View current subscription plan and status.
- Update payment method (provider-dependent).
- Schedule unsubscription (`cancel_at_period_end` style behavior).
- Retry failed or pending payment flows.

3.5 Important Behavior: Unsubscribe

Unsubscribing does not instantly terminate active service.

- Access remains valid until active period end.
- Cancellation effective date is displayed in settings/billing state.

3.6 Callback and Confirmation Behavior

Success callback page performs confirmation:

- Stripe confirmation using `stripe_session_id`.
- Paystack confirmation using reference keys:
  - `reference`,
  - `trxref`,
  - `paystack_reference`.

If confirmation fails:

- User can retry confirmation from the callback page.
- User can resume checkout if provider returns a resume URL.

3.7 Billing API Surfaces (User-Relevant)

- `GET /api/billing/health/`
- `GET /api/billing/exchange-rate/`
- `GET /api/billing/quotas/`
- `GET/PATCH/DELETE /api/billing/subscriptions/manage/`
- `POST /api/billing/subscriptions/manage/payment-method/update-session/`
- `POST /api/billing/subscriptions/manage/retry/`
- `POST /api/billing/subscriptions/confirm/`
- `POST /api/billing/subscriptions/access/verify/`
- Stripe:
  - `POST /api/billing/subscriptions/stripe/checkout-session/`
  - `POST /api/billing/subscriptions/stripe/confirm/`
  - `POST /api/billing/subscriptions/stripe/webhook/`
- Paystack:
  - `POST /api/billing/subscriptions/paystack/checkout-session/`
  - `POST /api/billing/subscriptions/paystack/confirm/`
  - `POST /api/billing/subscriptions/paystack/webhook/`

3.8 Multi-Currency Notes

- Stripe is typically USD-centric in this project flow.
- Paystack can operate with local currency configurations.
- Exchange-rate endpoint is available for conversion-aware UI and billing display logic.

3.9 Troubleshooting Billing Flow

If callback page hangs or fails:

1. Confirm callback URL includes session/reference parameter.
2. Check backend billing confirmation endpoint response.
3. Inspect provider webhook delivery status.
4. Use manual confirm endpoint for reconciliation.
5. Verify `STRIPE_*`, `PAYSTACK_*`, and exchange-rate env configuration.

---

4) Authentication, Login, and Security

4.1 Authentication Endpoints

Primary authentication APIs:

- `POST /api/auth/register/`
- `POST /api/auth/login/`
- `POST /api/auth/logout/`
- `POST /api/auth/admin/login/`
- `POST /api/auth/login/verify/`
- `POST /api/auth/admin/login/verify/`
- `POST /api/auth/token/refresh/` (if enabled)

4.2 Login Flow (Standard User)

1. Enter credentials on `/login`.
2. If 2FA is required, user is redirected to `/login/2fa`.
3. User submits OTP or backup code.
4. System issues authenticated session/JWT context.
5. Profile fetch runs automatically for role and permissions.

4.3 Admin Login Flow

Admin login uses dedicated backend route:

- `POST /api/auth/admin/login/`

2FA verification can still be required:

- `POST /api/auth/admin/login/verify/`

4.4 Password Management

Supported actions:

- Change password: `POST /api/auth/change-password/`
- Request reset: `POST /api/auth/password-reset/`
- Confirm reset: `POST /api/auth/password-reset-confirm/`

UI paths:

- `/change-password`

- `/forgot-password`
- `/forgot-password/email-sent`
- `/reset-password/:token`

4.5 Profile Management

Profile APIs:

- `GET /api/auth/profile/`
- `PATCH /api/auth/profile/update/`

User settings page supports:

- Personal details,
- Professional metadata,
- Optional profile fields,
- Billing management section (for non-applicant roles),
- Security action links.

4.6 Two-Factor Authentication (2FA)

2FA APIs:

- `POST /api/auth/admin/2fa/setup/`
- `POST /api/auth/admin/2fa/enable/`
- `GET /api/auth/2fa/status/`
- `POST /api/auth/2fa/backup-codes/regenerate/`

Operational model:

- Non-candidate accounts are expected to use 2FA.
- Backup codes are generated and stored securely (hashed).
- Backup code regeneration should be treated as a sensitive action.

4.7 Security Page

From `/security` (non-applicant roles):

- View 2FA status,
- Setup/enable authenticator flow,
- Regenerate backup codes,
- Validate account protection state.

4.8 Session and CSRF Notes

Common causes of login failure:

- Missing CSRF token on POST requests,
- Origin not present in trusted CSRF origins,
- Expired session or stale token after environment changes.

Best practices:

- Keep frontend origin in `CSRF_TRUSTED_ORIGINS`.
- Keep frontend origin in `CORS_ALLOWED_ORIGINS`.
- Use consistent protocol and hostnames across frontend/backend.

4.9 Email Delivery Behavior

Configured behavior:

- Debug/development: console/terminal email backend.
- Production: SMTP backend.

Ensure SMTP env values are configured in production:

- `EMAIL_HOST`
- `EMAIL_PORT`
- `EMAIL_USE_TLS`
- `EMAIL_HOST_USER`
- `EMAIL_HOST_PASSWORD`
- `DEFAULT_FROM_EMAIL`

---

5) HR Campaign and Rubric Workflows

5.1 Campaign Lifecycle Overview

Campaigns are the orchestration unit for vetting activities.

Core actions:

1. Create campaign.
2. Configure rubric version(s).
3. Import or enroll candidates.
4. Monitor dashboard progress.
5. Execute review and decisions.

5.2 Campaign Management Surfaces

Frontend pages:

- `/campaigns`
- `/campaigns/:campaignId`

Core APIs:

- `GET/POST /api/campaigns/`
- `GET/PATCH/DELETE /api/campaigns/{id}/`
- `POST /api/campaigns/{id}/candidates/import/`
- `GET /api/campaigns/{id}/dashboard/`
- `POST /api/campaigns/{id}/rubrics/versions/`
- `POST /api/campaigns/{id}/rubrics/versions/activate/`

5.3 Creating a Campaign

Recommended input data:

- Campaign name and purpose,
- Start/end windows,
- Ownership and team context,
- Operational settings for invitations and review.

Post-create checks:

- Campaign appears in list with editable state.
- Dashboard endpoint returns zeroed but valid summary.

5.4 Rubric Strategy

Rubrics define weighted scoring and evaluation criteria.

Pages:

- `/rubrics`
- `/rubrics/new`
- `/rubrics/:rubricId/edit`

Rubric APIs:

- `GET/POST /api/rubrics/vetting-rubrics/`
- `GET/PATCH/DELETE /api/rubrics/vetting-rubrics/{id}/`
- `POST /api/rubrics/vetting-rubrics/{id}/activate/`
- `POST /api/rubrics/vetting-rubrics/{id}/criteria/`
- `POST /api/rubrics/vetting-rubrics/{id}/duplicate/`
- `POST /api/rubrics/vetting-rubrics/create_from_template/`
- `GET /api/rubrics/vetting-rubrics/templates/`

5.5 Rubric Design Best Practices

1. Keep criteria explicit and measurable.
2. Use balanced weights; avoid over-concentrating one signal.
3. Define manual-review thresholds intentionally.
4. Validate rubric behavior on sample cases before activation.
5. Keep naming conventions stable for team consistency.

5.6 Evaluation Workflow

Case evaluation APIs:

- `POST /api/rubrics/vetting-rubrics/{id}/evaluate-case/`
- `POST /api/rubrics/vetting-rubrics/{id}/evaluate_application/`

Evaluation management APIs:

- `GET /api/rubrics/evaluations/`
- `GET /api/rubrics/evaluations/{id}/`
- `POST /api/rubrics/evaluations/{id}/rerun/`
- `POST /api/rubrics/evaluations/{id}/override-criterion/`

5.7 HR Daily Workflow Example

1. Open campaign workspace.
2. Confirm active rubric version.
3. Review intake totals (enrolled, in-progress, completed).
4. Investigate flagged candidates.
5. Trigger or rerun evaluation where needed.
6. Apply final human decision with rationale.

5.8 Common Campaign Mistakes

- Activating campaign without active rubric.
- Importing candidates before verifying quota.
- Using ambiguous rubric criteria descriptions.
- Ignoring manual-review flags for low-confidence outputs.

---

6) Candidate Invitations and Access Journey

6.1 Invitation Model

Candidates are onboarded through invitation flows rather than unrestricted public signups.

Main invitation APIs:

- `GET/POST /api/invitations/`
- `GET/PATCH/DELETE /api/invitations/{id}/`
- `POST /api/invitations/{id}/send/`
- `POST /api/invitations/accept/`

6.2 Candidate Access APIs

- `POST /api/invitations/access/consume/`
- `GET /api/invitations/access/me/`
- `GET /api/invitations/access/results/`
- `POST /api/invitations/access/logout/`

6.3 Candidate Entry Points

Frontend pages:

- `/invite/:token`
- `/candidate/access`

Typical sequence:

1. Candidate receives invitation link via channel.
2. Candidate accepts invitation token.
3. Candidate access session is consumed/bootstrapped.
4. Candidate sees allowed tasks/status.
5. Candidate submits required artifacts.
6. Candidate revisits results route after processing/decision.

6.4 Candidate Enrollment Surfaces

APIs:

- `GET/POST /api/enrollments/`
- `GET/PATCH/DELETE /api/enrollments/{id}/`
- `POST /api/enrollments/{id}/mark-complete/`

Related candidate APIs:

- `GET/POST /api/candidates/`
- `GET/PATCH/DELETE /api/candidates/{id}/`
- `GET/POST /api/social-profiles/`
- `GET/PATCH/DELETE /api/social-profiles/{id}/`

6.5 Invitation Operational Tips

1. Confirm candidate email and phone data quality before sending.
2. Re-send invitation if status indicates delivery failure or expiry.
3. Track acceptance timestamps for pipeline health.
4. Avoid issuing duplicate active invitations for same enrollment.

6.6 Candidate Session Security Notes

- Access flows are token/session scoped.
- Sessions can be explicitly closed via logout endpoint.
- Candidate privileges are intentionally narrower than staff/admin accounts.

6.7 Candidate Experience Checklist

From HR perspective, verify candidate can:

- Open invitation link,
- Authenticate into access session,
- Upload required documents,
- Complete interview stage if required,
- View result summary once published.

---

7) Applications and Document Vetting

7.1 Application Pages

Frontend routes:

- `/applications`
- `/applications/new`
- `/applications/:caseId`
- `/applications/:caseId/upload`

7.2 Application APIs

- `GET/POST /api/applications/cases/`
- `GET/PATCH/DELETE /api/applications/cases/{id}/`
- `POST /api/applications/cases/{id}/upload-document/`
- `GET /api/applications/cases/{id}/verification-status/`
- `POST /api/applications/cases/{id}/recheck-social-profiles/`

Document APIs:

- `GET/POST /api/applications/documents/`
- `GET/PATCH/DELETE /api/applications/documents/{id}/`

7.3 Document Vetting Components

The platform supports:

- OCR extraction,
- Authenticity scoring,
- Fraud risk scoring,
- Consistency checking,
- Social-profile consistency signals (where enabled).

7.4 Typical Case Workflow

1. Case created for a candidate.
2. Candidate uploads document set.
3. Case enters queued/processing analysis state.
4. Results are persisted and exposed in case detail.
5. HR reviews evidence and decides or escalates.

7.5 Verification Status Interpretation

Case status endpoint returns progress and outcomes that can include:

- pending/in-progress analysis,
- completed scoring outputs,
- flagged anomalies requiring human review.

Always treat low-confidence or conflicting signals as manual-review candidates.

7.6 Recheck Operations

Recheck endpoints can be used to:

- rerun social profile checks after profile updates,
- refresh stale analysis outputs,
- recover from transient provider failures.

7.7 Upload Guidance

To reduce verification errors:

1. Upload clear, readable, complete documents.
2. Avoid cropped corners or obscured IDs.
3. Keep file formats and naming conventions consistent.
4. Ensure candidate metadata matches submitted document identity fields.

7.8 Decision Quality Best Practices

Before final decision:

- Review both summary score and criterion-level outputs.
- Cross-check authenticity, fraud, and consistency outputs.
- Consider background check and interview context where available.
- Record rationale for overrides/escalations.

---

8) AI Interviews and Live Video Calls

8.1 Interview Surfaces

Frontend routes:

- `/interview/interrogation/:applicationId`
- `/video-calls`

Interview APIs:

- `GET/POST /api/interviews/sessions/`
- `GET/PATCH/DELETE /api/interviews/sessions/{id}/`
- `POST /api/interviews/sessions/{id}/start/`
- `POST /api/interviews/sessions/{id}/complete/`
- `POST /api/interviews/sessions/{id}/avatar-session/`
- `GET /api/interviews/sessions/{id}/playback/`
- `POST /api/interviews/sessions/{id}/save-exchange/`
- `POST /api/interviews/sessions/{id}/update-exchange/`
- analytics utility endpoints under `/api/interviews/sessions/*`

Interview content APIs:

- `GET/POST /api/interviews/questions/`
- `GET/POST /api/interviews/responses/`
- `POST /api/interviews/responses/{id}/analyze/`

- `GET/POST /api/interviews/feedback/`

8.2 Interview Flow

1. Session is scheduled or started.
2. Questions are asked/generated.
3. Responses are recorded/uploaded.
4. Response analysis is performed.
5. Session is completed and feedback is attached.

8.3 Video Meeting APIs (LiveKit-Oriented)

- `GET/POST /api/video-calls/meetings/`
- `GET/PATCH/DELETE /api/video-calls/meetings/{id}/`
- `POST /api/video-calls/meetings/{id}/start/`
- `POST /api/video-calls/meetings/{id}/complete/`
- `POST /api/video-calls/meetings/{id}/cancel/`
- `POST /api/video-calls/meetings/{id}/reschedule/`
- `POST /api/video-calls/meetings/{id}/extend/`
- `POST /api/video-calls/meetings/{id}/join-token/`
- `GET /api/video-calls/meetings/{id}/events/`
- `GET /api/video-calls/meetings/reminder-health/`
- recurring series endpoints:
  - `POST /api/video-calls/meetings/schedule-series/`
  - `POST /api/video-calls/meetings/{id}/reschedule-series/`
  - `POST /api/video-calls/meetings/{id}/cancel-series/`

8.4 Scheduling Best Practices

1. Choose clear timezone and meeting title conventions.
2. Confirm candidates receive invite notifications early.
3. Use extension/reschedule actions instead of deleting records.
4. Track reminder runtime health if reminder delivery appears delayed.

8.5 1v1 and 1vMany Notes

The meeting model supports structured scheduling and participant states.

- 1v1: HR and a single candidate.
- 1vMany: HR with multiple candidate participants.

Always confirm participant list and role metadata before start.

8.6 Troubleshooting Interview/Meeting Issues

If join fails:

1. Confirm join-token endpoint returns valid token.
2. Check LiveKit env configuration (`LIVEKIT_URL`, API key/secret).
3. Verify meeting state is startable (not canceled/completed).
4. Check reminder-health endpoint for runtime degradation.

---

9) Admin Operations and Control Center

9.1 Admin UI Areas

Admin-exclusive routes:

- `/admin/dashboard`
- `/admin/analytics`
- `/admin/register`
- `/admin/control-center`
- `/admin/users`
- `/admin/cases`
- `/admin/cases/:caseId`
- `/admin/rubrics`

9.2 Admin APIs

Admin dashboard endpoints:

- `GET /api/admin/dashboard/`
- `GET /api/admin/analytics/`
- `GET /api/admin/cases/`
- `GET /api/admin/users/`
- `PATCH /api/admin/users/{user_id}/`

9.3 Key Admin Responsibilities

1. Platform-level user management.
2. Global case oversight and intervention.
3. Policy and quality governance.
4. Operational visibility and escalation.
5. Access and permission lifecycle control.

9.4 User Management Procedures

For each user update:

1. Validate reason for change.
2. Confirm role impact (admin/hr/applicant).
3. Apply change and verify route/access behavior.
4. Log and communicate action where required.

9.5 Case Oversight Procedures

Admin case review focuses on:

- unresolved flagged cases,
- edge-case decisions requiring override,
- cross-workspace anomalies or abuse patterns,
- incident-triggered investigations.

9.6 Governance Recommendations

- Keep admin account count minimal.
- Enforce 2FA across all non-candidate accounts.
- Review audit logs routinely.
- Require rationale for sensitive state changes.

9.7 Delegation Model

Admins should avoid day-to-day vetting execution when possible.

- HR managers run campaign execution.
- Admins focus on:
  - policy enforcement,
  - platform stability,
  - compliance and forensic readiness.

---

10) Monitoring, Audit Logs, and AI Runtime Health

10.1 Monitoring Surfaces

Frontend pages:

- `/audit-logs` (admin only)
- `/ml-monitoring` (admin only)
- `/ai-monitor` (admin only)

Backend health route:

- `/api/system/health/`

10.2 Audit Endpoints

- `GET /api/audit/logs/`
- `GET /api/audit/logs/{id}/`
- `GET /api/audit/logs/by_entity/`
- `GET /api/audit/logs/recent_activity/`
- `GET /api/audit/logs/statistics/`

10.3 ML Monitoring Endpoints

- `GET /api/ml-monitoring/`
- `GET /api/ml-monitoring/{id}/`
- `GET /api/ml-monitoring/latest/`
- `GET /api/ml-monitoring/history/`
- `GET /api/ml-monitoring/performance-summary/`

Legacy aliases under `/api/ml-monitoring/metrics/` are also supported.

10.4 AI Monitor Endpoints

- `GET /api/ai-monitor/health/`
- `POST /api/ai-monitor/classify-document/`
- `POST /api/ai-monitor/check-social-profiles/`

10.5 Runtime Health Signals

Track:

- Request latency and error rates,
- Celery queue backlog and retry volume,
- Background check refresh failures,
- Billing confirmation lag,
- Video call reminder runtime availability.

10.6 Audit Usage Pattern

Use audit for:

1. Investigating disputed decisions.
2. Tracking high-risk role changes.
3. Reconstructing operational incidents.
4. Compliance and reporting evidence.

10.7 Monitoring Best Practices

1. Monitor trends, not only snapshots.
2. Alert on deviation thresholds.
3. Correlate user-facing incidents with task/runtime logs.
4. Keep service tokens and admin-only metrics endpoints protected.

10.8 Troubleshooting "Unavailable Runtime" Indicators

If runtime cards show unavailable:

1. Confirm backend health endpoint status.
2. Confirm Redis and Celery services are running.
3. Check endpoint-specific auth/permission context.
4. Inspect backend and worker logs for recent errors.

---

11) Background Checks and Provider Integration

11.1 Background Check Scope

OVS supports background checks as a managed subsystem with provider abstraction.

Key routes:

- Frontend: `/background-checks`
- APIs:
  - `GET/POST /api/background-checks/checks/`
  - `GET/PATCH/DELETE /api/background-checks/checks/{id}/`
  - `GET /api/background-checks/checks/{id}/events/`
  - `POST /api/background-checks/checks/{id}/refresh/`
  - `POST /api/background-checks/providers/{provider_key}/webhook/`

11.2 Provider Modes

Typical provider mode values:

- `mock` (development/testing only),
- `http` (real external provider integration).

Production recommendation:

- Use real provider mode and enforce webhook/token security.

11.3 Consent and Governance

Background check operations should require:

- explicit candidate consent records,
- traceable run metadata,
- status progression with reviewability.

11.4 Refresh and Event Flow

1. Create or trigger check.
2. Provider submission occurs.
3. Events and status updates are tracked.
4. Refresh endpoint can pull latest provider status.
5. Webhooks update state asynchronously.

11.5 Webhook Security Practices

1. Validate provider-specific signatures or auth headers.
2. Keep webhook secret/token in secured environment variables.
3. Ensure idempotency when processing repeated events.
4. Log every webhook payload metadata for forensics.

11.6 Operator Checklist

Before enabling production background checks:

1. Set provider base URL.
2. Set provider API key.
3. Set webhook token/secret.
4. Validate webhook endpoint reachability.
5. Test submit -> refresh -> final status path.

11.7 Common Background Check Issues

- Provider returns partial data: treat as pending, not final fail.
- Webhook not received: use refresh endpoint and provider logs.
- Unexpected status mapping: verify provider adapter and normalization logic.

---

12) Notifications and Decision Lifecycle

12.1 Notification Surfaces

Frontend route:

- `/notifications`

Notification APIs:

- `GET /api/notifications/`
- `GET /api/notifications/{id}/`
- `POST /api/notifications/{id}/mark_read/`
- `DELETE /api/notifications/{id}/archive/`
- `POST /api/notifications/mark-as-read/`
- `POST /api/notifications/mark-all-as-read/`
- `GET /api/notifications/unread-count/`

12.2 Notification Use Cases

- Invitation delivery and reminders,
- Case progress updates,
- Interview scheduling/reminders,
- Billing/subscription state changes,
- Decision publication to candidates.

12.3 Decision Lifecycle Model

Typical status progression:

1. Candidate invited/enrolled.
2. Evidence collection.
3. AI analysis and score aggregation.
4. HR/Admin review.
5. Final decision:
    - approve,
    - reject,
    - escalate/manual review.
6. Candidate and stakeholders notified.

12.4 Decision Quality Controls

Before finalizing:

- Confirm rubric context and active version.
- Check for unresolved high-risk flags.
- Ensure all required evidence has completed processing.
- Add reviewer notes where required by policy.

12.5 Notification Hygiene

Recommended team process:

1. Keep unread count near zero for operational roles.
2. Archive obsolete notifications to reduce noise.
3. Use filters and case links for triage.
4. Correlate decision notifications with audit entries.

---

13) Operational Procedures (Docker, Commands, and Checks)

13.1 Standard Startup (Docker)

From project root:

```
docker compose up -d --build
```

Validate service state:

```
docker compose ps
```

Expected core services:

- `db`
- `redis`

- backend
- celery_worker
- celery_beat
- flower

13.2 Migrations and Admin Bootstrap

```
docker compose exec backend python manage.py migrate
docker compose exec backend python manage.py createsuperuser
```

13.3 Health and Integrity Checks

```
docker compose exec backend python manage.py check --settings=config.setti
docker compose exec backend python manage.py check_uuid_schema
```

13.4 AI/ML Command Utilities

Command entrypoint examples:

```
docker compose exec backend python manage.py check_ai_ml_services
docker compose exec backend python manage.py check_ai_ml_services --strict
docker compose exec backend python manage.py generate_model_manifest
docker compose exec backend python manage.py train_ai_models
docker compose exec backend python manage.py train_document_classifiers
```

13.5 Logging During Operations

Stream logs:

```
docker compose logs -f backend
docker compose logs -f celery_worker
docker compose logs -f celery_beat
```

13.6 Release Gate Commands (Recommended)

Backend:

```
docker compose exec backend python manage.py test --keepdb
docker compose exec backend python manage.py check --deploy --settings=con
```

Frontend:

```
npm --prefix frontend run lint
npm --prefix frontend run type-check
npm --prefix frontend run test
npm --prefix frontend run build:ci
npm --prefix frontend run coverage:endpoints -- --strict
```

13.7 Email Mode Validation

In development, confirm console email behavior:

```
docker compose exec backend python manage.py shell -c "from django.conf im
```

Expected in debug/development:

- `django.core.mail.backends.console.EmailBackend`

Expected in production settings:

- `django.core.mail.backends.smtp.EmailBackend`

13.8 Production Compose Notes

Use dedicated production compose file:

```
docker compose -f docker-compose.prod.yml up -d
```

Production prerequisites:

- Hosted PostgreSQL `DATABASE_URL`,
- HTTPS origin settings,
- provider secrets (billing/background checks),
- rotated and secured credentials.

---

14) Troubleshooting Guide

14.1 Login and Auth Problems

Symptom: "CSRF token missing" or origin check failed

Actions:

1. Confirm frontend origin is in `CSRF_TRUSTED_ORIGINS`.
2. Confirm frontend origin is in `CORS_ALLOWED_ORIGINS`.
3. Restart backend after env changes.
4. Clear stale browser cookies/session and retry.

Symptom: Login succeeds but redirects incorrectly

Actions:

1. Confirm user role in profile payload.
2. Confirm route guard logic has expected user type.
3. Verify profile fetch completes after login.

14.2 Billing/Checkout Problems

Symptom: `/billing/success` hangs on "Confirming Payment"

Actions:

1. Confirm success URL contains session/reference parameter.
2. Check browser network for confirm endpoint call.
3. Call confirm endpoint manually for diagnosis.
4. Check backend billing logs and provider dashboard.

Symptom: "Transaction reference not found"

Actions:

1. Use exact reference returned by latest checkout session.
2. Ensure confirm call matches the provider (Stripe vs Paystack).
3. Ensure backend env keys target same provider account mode.

14.3 Reminder Runtime Unavailable

Symptom: reminder runtime card shows unavailable or fetch error

Actions:

1. Confirm backend and Celery services are healthy.
2. Confirm reminder endpoint permissions for current user.
3. Review worker logs for failed reminder jobs.
4. Verify Redis connectivity.

14.4 Candidate/Invitation Issues

Symptom: candidate cannot access invite link

Actions:

1. Verify invitation token and expiry state.
2. Re-send invitation from invitation endpoint.
3. Confirm candidate access consume endpoint works.
4. Check candidate enrollment status.

14.5 Video Call Join Issues

Symptom: cannot join meeting

Actions:

1. Confirm meeting is started or joinable.
2. Verify join-token endpoint response.
3. Verify LiveKit env configuration.
4. Check network/firewall restrictions for realtime transport.

14.6 Background Check Stuck in Pending

Actions:

1. Trigger refresh endpoint.
2. Confirm provider API credentials and base URL.
3. Verify webhook path and token/secret validation.
4. Inspect provider-side event logs.

14.7 AI Monitoring Endpoints Not Accessible

Actions:

1. Confirm account is admin/staff where required.
2. Confirm service token paths are configured as intended.
3. Validate endpoint path and auth headers.

14.8 Docker Build/Startup Issues

Symptom: image build fails or service unhealthy

Actions:

1. Ensure `.dockerignore` excludes local virtual env and large datasets.

2. Rebuild with no cache if dependency corruption suspected:

```
docker compose build --no-cache
```

1. Start stack and inspect health:

```
docker compose up -d
docker compose ps
docker compose logs -f backend
```

14.9 Last-Resort Incident Procedure

1. Freeze user-facing writes if data integrity risk is suspected.
2. Collect logs from backend, worker, and provider callbacks.
3. Capture failing request/response payload metadata.
4. Reproduce issue in controlled environment.
5. Apply fix and verify with targeted test.

---

15) API Endpoint Quick Map

This is a quick operational map of the active backend API groups.
Base prefix: `/api/`

15.1 System and Core

- `/api/system/health/`

15.2 Authentication

- `/api/auth/register/`
- `/api/auth/login/`
- `/api/auth/logout/`
- `/api/auth/admin/login/`
- `/api/auth/login/verify/`
- `/api/auth/admin/login/verify/`
- `/api/auth/change-password/`
- `/api/auth/password-reset/`
- `/api/auth/password-reset-confirm/`
- `/api/auth/profile/`
- `/api/auth/profile/update/`
- `/api/auth/2fa/status/`
- `/api/auth/2fa/backup-codes/regenerate/`
- `/api/auth/admin/2fa/setup/`
- `/api/auth/admin/2fa/enable/`
- `/api/auth/token/refresh/` (optional when enabled)

15.3 Campaigns, Candidates, Enrollments, Invitations

- `/api/campaigns/`
- `/api/campaigns/{id}/`
- `/api/campaigns/{id}/candidates/import/`
- `/api/campaigns/{id}/dashboard/`
- `/api/campaigns/{id}/rubrics/versions/`

- /api/campaigns/{id}/rubrics/versions/activate/
- /api/candidates/
- /api/candidates/{id}/
- /api/enrollments/
- /api/enrollments/{id}/
- /api/enrollments/{id}/mark-complete/
- /api/invitations/
- /api/invitations/{id}/
- /api/invitations/{id}/send/
- /api/invitations/accept/
- /api/invitations/access/consume/
- /api/invitations/access/me/
- /api/invitations/access/results/
- /api/invitations/access/logout/
- /api/social-profiles/
- /api/social-profiles/{id}/

15.4 Applications and Documents

- /api/applications/cases/
- /api/applications/cases/{id}/
- /api/applications/cases/{id}/upload-document/
- /api/applications/cases/{id}/verification-status/
- /api/applications/cases/{id}/recheck-social-profiles/
- /api/applications/documents/
- /api/applications/documents/{id}/

15.5 Interviews and Video Calls

Interviews:

- /api/interviews/interrogation/start/
- /api/interviews/upload-response/
- /api/interviews/sessions/
- /api/interviews/sessions/{id}/
- /api/interviews/sessions/{id}/start/
- /api/interviews/sessions/{id}/complete/
- /api/interviews/sessions/{id}/avatar-session/
- /api/interviews/sessions/{id}/playback/
- /api/interviews/sessions/{id}/save-exchange/
- /api/interviews/sessions/{id}/update-exchange/
- /api/interviews/sessions/analytics-dashboard/
- /api/interviews/sessions/compare/
- /api/interviews/sessions/generate-flags/
- /api/interviews/questions/
- /api/interviews/questions/{id}/
- /api/interviews/responses/
- /api/interviews/responses/{id}/
- /api/interviews/responses/{id}/analyze/
- /api/interviews/feedback/

- /api/interviews/feedback/{id}/

Video calls:

- /api/video-calls/meetings/
- /api/video-calls/meetings/{id}/
- /api/video-calls/meetings/upcoming/
- /api/video-calls/meetings/reminder-health/
- /api/video-calls/meetings/schedule-series/
- /api/video-calls/meetings/{id}/start/
- /api/video-calls/meetings/{id}/complete/
- /api/video-calls/meetings/{id}/cancel/
- /api/video-calls/meetings/{id}/reschedule/
- /api/video-calls/meetings/{id}/extend/
- /api/video-calls/meetings/{id}/leave/
- /api/video-calls/meetings/{id}/join-token/
- /api/video-calls/meetings/{id}/events/
- /api/video-calls/meetings/{id}/calendar-ics/
- /api/video-calls/meetings/{id}/cancel-series/
- /api/video-calls/meetings/{id}/reschedule-series/

15.6 Rubrics and Scoring

- /api/rubrics/vetting-rubrics/
- /api/rubrics/vetting-rubrics/{id}/
- /api/rubrics/vetting-rubrics/{id}/activate/
- /api/rubrics/vetting-rubrics/{id}/criteria/
- /api/rubrics/vetting-rubrics/{id}/duplicate/
- /api/rubrics/vetting-rubrics/{id}/evaluate-case/
- /api/rubrics/vetting-rubrics/{id}/evaluate_application/
- /api/rubrics/vetting-rubrics/create_from_template/
- /api/rubrics/vetting-rubrics/templates/
- /api/rubrics/criteria/
- /api/rubrics/criteria/{id}/
- /api/rubrics/evaluations/
- /api/rubrics/evaluations/{id}/
- /api/rubrics/evaluations/{id}/rerun/
- /api/rubrics/evaluations/{id}/override-criterion/

15.7 Notifications, Audit, Fraud, Monitoring

Notifications:

- /api/notifications/
- /api/notifications/{id}/
- /api/notifications/{id}/mark_read/
- /api/notifications/{id}/archive/
- /api/notifications/mark-as-read/
- /api/notifications/mark-all-as-read/
- /api/notifications/unread-count/

Audit:

- `/api/audit/logs/`
- `/api/audit/logs/{id}/`
- `/api/audit/logs/by_entity/`
- `/api/audit/logs/recent_activity/`
- `/api/audit/logs/statistics/`

Fraud:

- `/api/fraud/results/`
- `/api/fraud/results/{id}/`
- `/api/fraud/results/statistics/`
- `/api/fraud/consistency/`
- `/api/fraud/consistency/{id}/`
- `/api/fraud/consistency/history/`
- `/api/fraud/consistency/statistics/`
- `/api/fraud/social-profiles/`
- `/api/fraud/social-profiles/{id}/`
- `/api/fraud/social-profiles/statistics/`

ML Monitoring:

- `/api/ml-monitoring/`
- `/api/ml-monitoring/{id}/`
- `/api/ml-monitoring/latest/`
- `/api/ml-monitoring/history/`
- `/api/ml-monitoring/performance-summary/`
- Legacy alias group: `/api/ml-monitoring/metrics/...`

AI Monitor:

- `/api/ai-monitor/health/`
- `/api/ai-monitor/classify-document/`
- `/api/ai-monitor/check-social-profiles/`

15.8 Billing and Background Checks

Billing:

- `/api/billing/health/`
- `/api/billing/exchange-rate/`
- `/api/billing/quotas/`
- `/api/billing/subscriptions/manage/`
- `/api/billing/subscriptions/manage/payment-method/update-session/`
- `/api/billing/subscriptions/manage/retry/`
- `/api/billing/subscriptions/confirm/`
- `/api/billing/subscriptions/access/verify/`
- `/api/billing/subscriptions/stripe/checkout-session/`
- `/api/billing/subscriptions/stripe/confirm/`
- `/api/billing/subscriptions/stripe/webhook/`
- `/api/billing/subscriptions/paystack/checkout-session/`
- `/api/billing/subscriptions/paystack/confirm/`
- `/api/billing/subscriptions/paystack/webhook/`

Background checks:

- `/api/background-checks/checks/`
- `/api/background-checks/checks/{id}/`
- `/api/background-checks/checks/{id}/events/`
- `/api/background-checks/checks/{id}/refresh/`
- `/api/background-checks/providers/{provider_key}/webhook/`

---

16) Glossary and Best Practices

16.1 Glossary

- Campaign: A vetting process container with rules, candidates, and timeline.
- Rubric: Weighted scoring template used to evaluate case evidence.
- Enrollment: Candidate linkage to a specific campaign.
- Invitation: Tokenized candidate onboarding entry.
- Case/Application: Candidate evidence package for review.
- Verification status: Current state of analysis and review for a case.
- Manual review: Human-required decision path for uncertain AI outputs.
- Subscription ticket/access verification: Billing-linked authorization to register or continue service.
- Reminder runtime: Scheduler/worker state for meeting reminders and notifications.
- Provider webhook: External callback notifying payment/background check status updates.
- Reconciliation: Explicit status refresh/confirm call to recover missed callback events.

16.2 User Best Practices

For HR Managers

1. Activate rubric before importing candidates.
2. Validate data quality during import.
3. Review low-confidence cases manually.
4. Keep campaign notes and decision rationale consistent.

For Admins

1. Limit admin accounts and enforce 2FA.
2. Review audit logs routinely.
3. Track operational metrics for backlog/failure trends.
4. Separate policy controls from day-to-day vetting execution.

For Candidates

1. Use valid invitation links only.
2. Submit clear and complete documents.
3. Complete interview steps within provided window.
4. Keep personal information accurate.

16.3 Operational Best Practices

1. Run full release gate before major deployments.
2. Use production-safe env configuration and secret management.
3. Keep logs available during critical operations and demos.
4. Test provider callbacks in staging before production rollout.
5. Maintain one deterministic fallback case for demos/training.

16.4 Security Best Practices

1. Do not commit secrets or API keys into source control.
2. Rotate exposed credentials immediately.
3. Keep CSRF/CORS origins explicit and least-privileged.
4. Restrict service-token usage to service-to-service contexts only.
5. Treat billing and background-check webhooks as high-integrity entry points.

16.5 Decision Governance Best Practices

1. Use AI as decision support, not absolute authority.
2. Preserve criterion-level evidence on every major decision.
3. Require manual review for conflicts or low-confidence outputs.
4. Log overrides with justification and reviewer identity.
5. Maintain clear retention and archival policies.

16.6 Manual Maintenance Policy

To keep this manual accurate:

1. Update route references whenever `frontend/src/App.tsx` changes.
2. Update endpoint map whenever `backend/openapi.yaml` changes.
3. Review role behavior whenever auth or permissions are modified.
4. Review billing and provider sections whenever integrations change.
5. Version user manual updates alongside release tags.

---

17) Frequently Asked Questions (FAQ)

17.1 Access and Accounts

Q: Can anyone sign up directly?

A: The recommended flow is subscription-first onboarding, then registration access based on successful subscription confirmation.

Q: Why am I redirected to 2FA after login?

A: Non-candidate accounts are expected to complete two-factor verification to reduce account takeover risk.

Q: Why can I not see admin pages?

A: Admin pages are role-guarded. Confirm your account `user_type` is `admin`.

17.2 Billing

Q: Why does cancellation not immediately remove access?

A: Unsubscribe operations are period-end cancellations by design. Service remains active until current billing period ends.

Q: Why does callback say payment pending even after checkout?

A: Provider status can lag. Use retry confirmation and verify webhook delivery or manual confirm endpoint response.

Q: Can payment method be changed?

A: Yes, through settings billing controls. Exact path depends on provider (Stripe hosted update session vs provider-specific flow).

17.3 Campaigns and Rubrics

Q: Who should create rubrics?

A: HR managers typically own rubric design per campaign. Admins can oversee governance.

Q: Can a rubric be changed after activation?

A: Use versioning patterns. Avoid silent in-place changes that compromise auditability.

Q: Why are some cases marked for manual review?

A: The system routes low-confidence or conflicting signals for human decision.

17.4 Candidate Journey

Q: Candidate says invitation link is invalid

A: Check token expiry, invitation state, and enrollment link. Re-send invitation if necessary.

Q: Can candidate have a permanent account?

A: Candidate flow is designed for scoped participation; persistent account behavior depends on your configured process policy.

17.5 Interviews and Video Calls

Q: Why is meeting join blocked?

A: Common causes include invalid meeting state, token generation failure, or LiveKit configuration issues.

Q: What is reminder runtime health?

A: It indicates readiness of scheduled reminder processing pipeline (worker/beat/dependencies).

17.6 Monitoring and Audit

Q: Who should access AI/ML monitoring?

A: Typically admins and authorized staff only.

Q: Why audit logs matter?

A: They preserve who did what and when, which is critical for accountability and incident reconstruction.

17.7 Background Checks

Q: Is background check provider integration real or mock?

A: It depends on configuration. Mock is suitable for development; production should use a real provider mode.

Q: Webhook isn’t updating status. What next?

A: Use refresh endpoint, inspect provider logs, then confirm webhook auth/signature settings.

---

18) Role Checklists and SOPs

18.1 Admin Daily Checklist

1. Verify service health and core metrics.
2. Review unread critical notifications.
3. Review audit anomalies and user-role changes.
4. Check billing/webhook error indicators.
5. Confirm no critical queue backlog.

18.2 HR Manager Daily Checklist

1. Open campaign dashboard and review active workloads.
2. Confirm pending invitations and follow-ups.
3. Review flagged/low-confidence cases.
4. Confirm interview schedules and reminders.
5. Close decision loop with rationale notes.

18.3 Candidate Support Checklist

For support staff handling candidate issues:

1. Validate invitation token state.
2. Confirm candidate enrollment status.
3. Confirm document upload availability.
4. Confirm interview session accessibility.
5. Confirm results publication state.

18.4 Billing SOP

1. Verify current subscription status.
2. Verify payment method state.
3. If failed/pending:
   - trigger retry flow,
   - confirm provider status,
   - reconcile with confirm endpoint.
4. If cancellation requested:
   - schedule period-end cancellation,
   - communicate effective date.

18.5 Incident SOP: Provider Outage

1. Declare degraded mode and notify operators.
2. Keep candidate-facing workflows available where possible.
3. Queue or defer provider-dependent operations.
4. Use fallback deterministic case for demos/training.
5. Reconcile delayed operations once provider recovers.

18.6 Incident SOP: Authentication Failures

1. Verify CSRF/CORS origin configuration.

2. Verify 2FA status and challenge path.
3. Verify token/session freshness.
4. Check login history for suspicious patterns.

18.7 Incident SOP: Data Integrity Concern

1. Pause high-risk mutating operations if needed.
2. Collect logs, request IDs, and payload metadata.
3. Identify affected entities and scope.
4. Validate DB records against expected workflow states.
5. Apply controlled recovery and verify through tests.

18.8 Release-Day Checklist

1. Execute full release gate command sequence.
2. Confirm production env variables.
3. Validate webhook endpoints and secrets.
4. Confirm SMTP delivery path in production.
5. Confirm rollback plan and backup availability.

18.9 Defense/Demo-Day Checklist

1. Run service health checks.
2. Confirm admin and HR login paths.
3. Confirm one complete fallback case is available.
4. Keep backend and worker logs visible.
5. Follow `DEFENSE_PACK.md` order if interrupted.